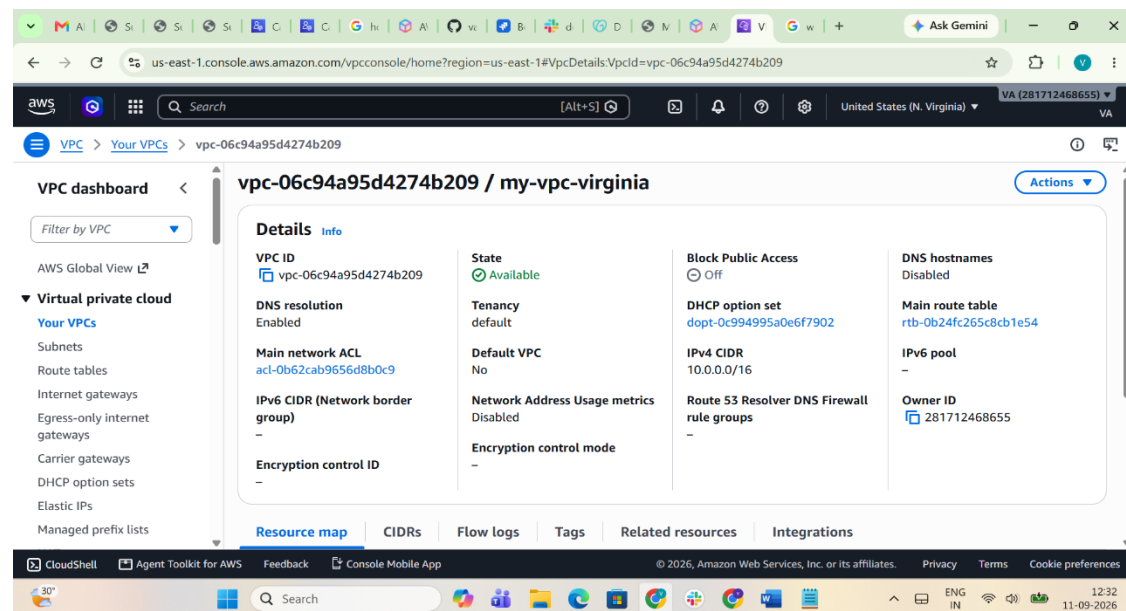


Task on ASG

1. Create one VPC in N. Virginia region.

Implementation

1. Open **AWS Console** → **VPC**.
2. Select **N. Virginia (us-east-1)**.
3. Go to **Your VPCs** → **Create VPC**.
4. Select **VPC only**.
5. Enter VPC name: **My-Project-VPC**.
6. Enter IPv4 CIDR: **10.0.0.0/16**.
7. Click **Create VPC**.



2. Create two subnets: one public subnet and one private subnet.

Implementation

1. Go to **VPC** → **Subnets** → **Create subnet**.
2. Select **My-Project-VPC**.
3. Create public subnet:
 - Name: **Public-Subnet**
 - CIDR: **10.0.1.0/24**
 - AZ: **us-east-1a**
5. Create private subnet:

6. Name: Private-Subnet

- CIDR: 10.0.2.0/24
- AZ: us-east-1a

For the public subnet, enable **Auto-assign public IPv4 address**.

The screenshot shows the AWS VPC console for a public subnet. The details panel includes the following information:

Property	Value
Subnet ID	subnet-0fdabd331e68229f2
Subnet ARN	arn:aws:ec2:us-east-1:281712468655:subnet/subnet-0fdabd331e68229f2
State	Available
Block Public Access	Off
IPv4 CIDR	10.0.0.0/25
Availability Zone	us-east-1a
Network ACL	-
Auto-assign customer-owned IPv4 address	No
IPv4 CIDR reservations	-
Available IPv4 addresses	123
Network border group	us-east-1
Default subnet	No
Customer-owned IPv4 pool	-
IPv6-only	No
VPC	vpc-06c94a95d4274b209 my-vpc-virginia
Auto-assign public IPv4 address	No
Outpost ID	-
Hostname type	IP name
Route table	rtb-0b24fc265c8cb1e54
Auto-assign IPv6 address	No
IPv4 CIDR reservations	-
Resource name DNS A record	Disabled

The screenshot shows the AWS VPC console for a private subnet. The details panel includes the following information:

Property	Value
Subnet ID	subnet-09403bff77bd60066
Subnet ARN	arn:aws:ec2:us-east-1:281712468655:subnet/subnet-09403bff77bd60066
State	Available
Block Public Access	Off
IPv4 CIDR	10.0.1.0/25
Availability Zone	us-east-1a
Network ACL	-
Auto-assign customer-owned IPv4 address	No
IPv4 CIDR reservations	-
Available IPv4 addresses	123
Network border group	us-east-1
Default subnet	No
Customer-owned IPv4 pool	-
IPv6-only	No
VPC	vpc-06c94a95d4274b209 my-vpc-virginia
Auto-assign public IPv4 address	No
Outpost ID	-
Hostname type	IP name
Route table	rtb-0b24fc265c8cb1e54
Auto-assign IPv6 address	No
IPv4 CIDR reservations	-
Resource name DNS A record	Disabled

Subnets (2) Info

Last updated less than a minute ago

Actions

Create subnet

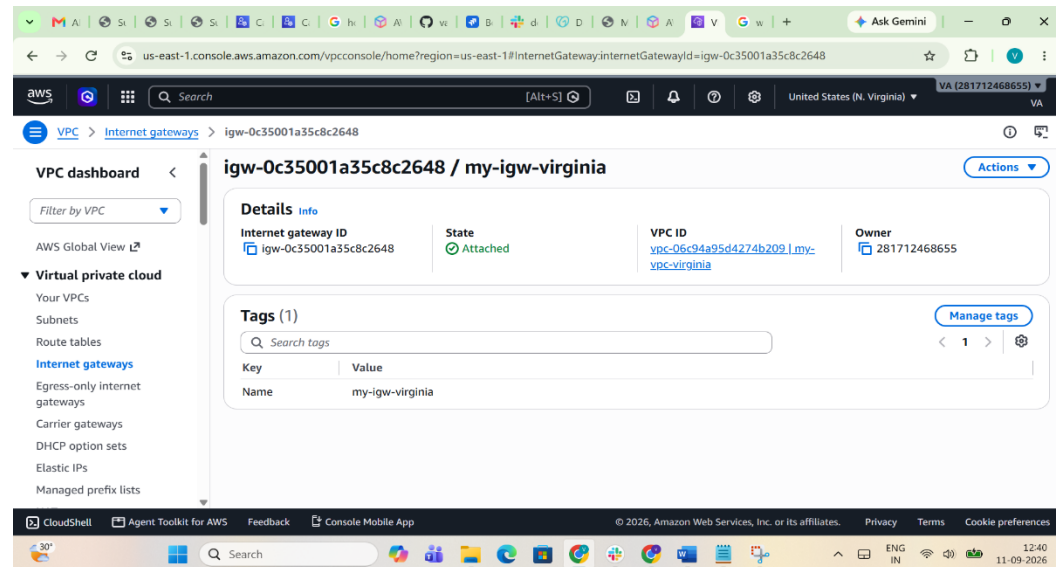
Find subnets by attribute or tag

Name	Subnet ID	State	VPC
public-subnet	subnet-0fdabd331e68229f2	Available	vpc-06c94a95d4274b209 my-vpc-virginia
private-subnet	subnet-09403bff77bd60066	Available	vpc-06c94a95d4274b209 my-vpc-virginia

3. Attach an IGW to the VPC.

Implementation

1. Go to **VPC → Internet Gateways**.
2. Click **Create internet gateway**.
3. Name it My-igw-virginia.
4. Create the IGW.
5. Select the IGW → **Actions → Attach to VPC**.
6. Select My-vpc-virginia.
7. Click **Attach internet gateway**.



4. Create one public route table (RT) and one private route table.

Implementation

Public Route Table

1. Go to **VPC → Route Tables → Create route table**.
2. Name: Public-RT.
3. Select My-vpc-virginia.
4. Create the route table.
5. Open **Routes → Edit routes**.
6. Add:
7. Destination: 0.0.0.0/0
8. Target: My-Project-IGW
9. Associate Public-Subnet with Public-RT.

Private Route Table

10. Create another route table.
11. Name: Private-RT.

12. Select My-vpc-virginia.
13. Create the route table.
14. Associate Private-Subnet with Private-RT.

The screenshot shows the AWS Management Console interface for a VPC named 'my-vpc-virginia'. The 'Route tables' section is selected, and the details for the route table 'rtb-0bacb2ab7c1743b45 / private-RT' are displayed. The 'Details' tab shows the route table ID, VPC, and owner ID. The 'Routes' tab shows a single route for the destination '10.0.0.0/16' with a target of 'local' and a status of 'Active'. The 'Subnet associations' tab shows the route table is associated with the 'private-subnet'.

The screenshot shows the AWS Management Console interface for a VPC named 'my-vpc-virginia'. The 'Route tables' section is selected, and the details for the route table 'rtb-0b24fc265c8cb1e54 / public-RT' are displayed. The 'Details' tab shows the route table ID, VPC, and owner ID. The 'Routes' tab shows a single route for the destination '0.0.0.0/0' with a target of 'igw-0c35001a35c8c2...' and a status of 'Active'. The 'Subnet associations' tab shows the route table is associated with the 'public-subnet'.

Route tables (2) Info						Last updated 3 minutes ago	Actions	Create route table
Find route tables by attribute or tag						< 1 >		
☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main			
☐	public-RT	rtb-0b24fc265c8cb1e54	subnet-0fdabd331e6822...	-	Yes			
☐	private-RT	rtb-0bacb2ab7c1743b45	subnet-09403bff77bd60...	-	No			

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Implementation

1. Go to VPC → NAT Gateways.

2. Click **Create NAT Gateway**.
3. Select Public-Subnet.
4. Select **Public** connectivity type.
5. Allocate an **Elastic IP**.
6. Click **Create NAT Gateway**.
7. Wait until the status becomes **Available**.
8. Open Private-RT.
9. Go to **Routes** → **Edit routes**.
10. Add:

- Destination: 0.0.0.0/0
- Target: NAT Gateway.

11. Save the route.

The screenshot shows the AWS Management Console for a NAT Gateway. The breadcrumb navigation is VPC > NAT gateways > nat-065b29322023934de. The left sidebar shows the 'Virtual private cloud' section with various resources. The main content area displays the details for the NAT Gateway 'nat-065b29322023934de / my-ngw-virginia'. The 'Details' tab is active, showing the NAT gateway ID, ARN, VPC, connectivity type (Public), primary public and private IPv4 addresses, subnet, and creation date. The state is 'Available'.

The screenshot shows the AWS Management Console for a Route Table. The breadcrumb navigation is VPC > Route tables > rtb-0bacb2ab7c1743b45. The left sidebar shows the 'Virtual private cloud' section. The main content area displays the details for the Route Table 'rtb-0bacb2ab7c1743b45'. The 'Routes' tab is active, showing a list of routes. The first route has a destination of 0.0.0.0/0, a target of 'nat-065b29322023934de', and a status of 'Active'. The second route has a destination of 10.0.0.0/16, a target of 'local', and a status of 'Active'.

6. Create two instances, one in the public subnet and one in the private subnet.

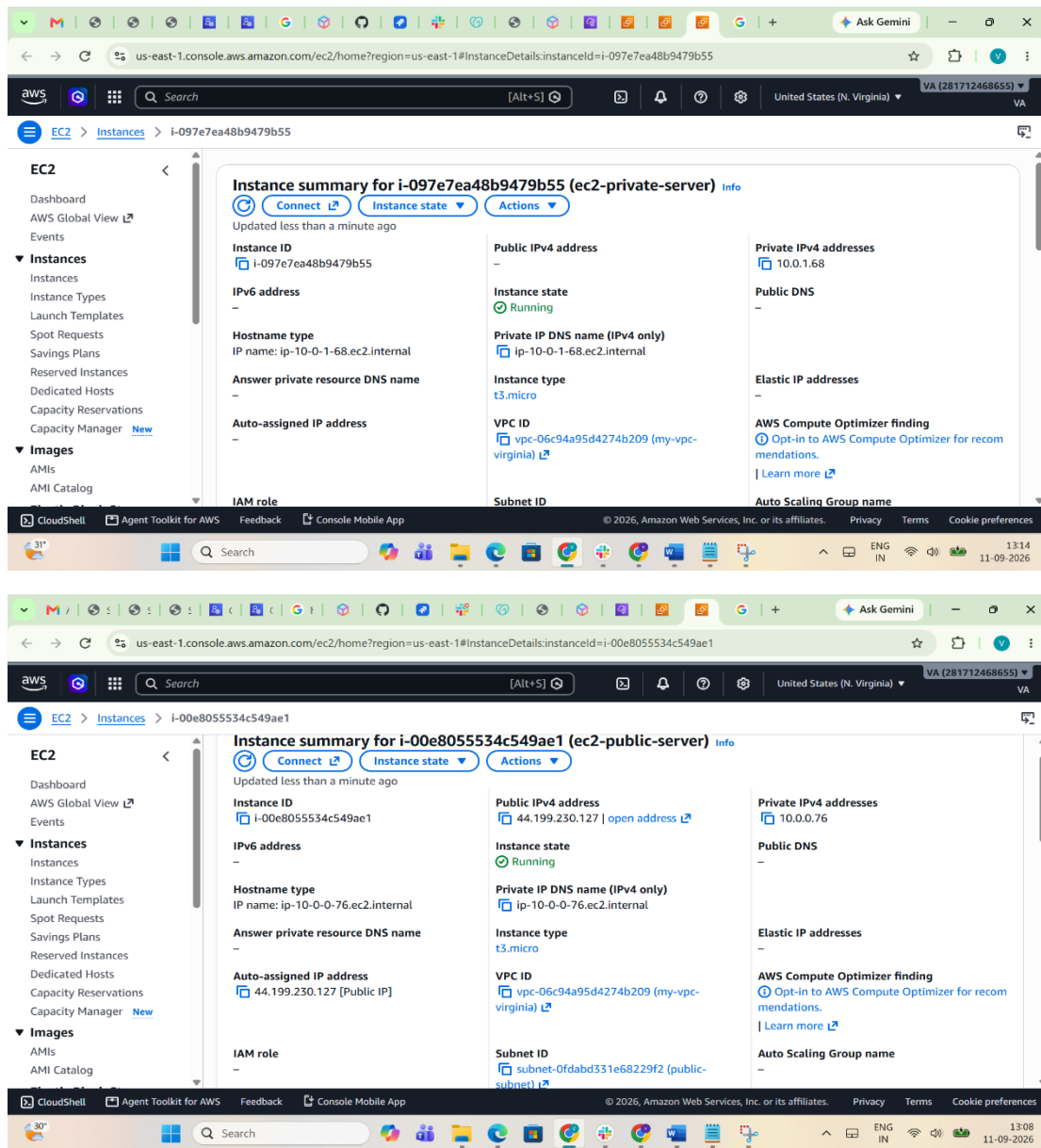
Implementation

Public EC2

1. Go to **EC2** → **Launch Instance**.
2. Name: ec2-Public-server.
3. Select **Amazon Linux 2023**.
4. Select t3.micro.
5. Select your key pair.
6. Select My-vpc-virginia.
7. Select Public-Subnet.
8. Enable **Auto-assign Public IP**.
9. Create/select a security group.
10. Allow SSH 22 from your IP.
11. Allow HTTP 80.
12. Launch the instance.

Private EC2

13. Launch another EC2 instance.
14. Name: ec2-Private-server.
15. Select **Amazon Linux 2023**.
16. Select t3.micro.
17. Select My-vpc-virginia.
18. Select Private-Subnet.
19. Disable public IP.
20. Allow HTTP 80 from the ALB security group.
21. Launch the instance.



7. Deploy Apache server on both EC2 instances with a sample index.html file.

Implementation

1. Connect to the public EC2 instance.
2. Install Apache: `sudo dnf install httpd -y`
3. Start Apache: `sudo systemctl start httpd`
4. Enable Apache: `sudo systemctl enable httpd`
5. Create the webpage: `sudo vi /var/www/html/index.html`
6. Add sample HTML content.
7. Restart Apache: `sudo systemctl restart httpd`

8. Verify: `sudo systemctl status httpd`
9. Repeat the same Apache installation on the private EC2.
10. Test the private server from an allowed internal source using: `curl http://<PRIVATE-IP>`.

```

$ ssh -i linux-testing.pem ec2-user@44.199.230.127
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Fri Sep 11 07:53:43 2026 from 103.203.172.230
[ec2-user@ip-10-0-0-76 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-0-76 ~]$ sudo su -
[root@ip-10-0-0-76 ~]# yum install httpd -y
Last metadata expiration check: 0:00:40 ago on Fri Sep 11 08:52:19 2026.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.68-1.amzn2023.0.1	amazonlinux	46 k
Installing dependencies:				
apr	x86_64	1.7.5-1.amzn2023.0.4	amazonlinux	129 k
apr-util	x86_64	1.6.5-1.amzn2023.0.1	amazonlinux	100 k
apr-util-ldap	x86_64	1.6.5-1.amzn2023.0.1	amazonlinux	15 k
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k

```

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_lua-2.4.68-1.amzn2023.0.1.x86_64
apr-util-1.6.5-1.amzn2023.0.1.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch
mailcap-2.1.49-3.amzn2023.0.3.noarch

Complete!
[root@ip-10-0-0-76 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-0-76 ~]# systemctl start httpd
[root@ip-10-0-0-76 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 08:54:11 UTC; 15s ago
     Docs: man:httpd.service(8)
  Main PID: 28410 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1059)
   Memory: 14.0M
      CPU: 72ms
   CGroup: /system.slice/httpd.service
           └─28410 /usr/sbin/httpd -DFOREGROUND
             └─28411 /usr/sbin/httpd -DFOREGROUND
               └─28412 /usr/sbin/httpd -DFOREGROUND
                 └─28413 /usr/sbin/httpd -DFOREGROUND
                   └─28416 /usr/sbin/httpd -DFOREGROUND

Sep 11 08:54:11 ip-10-0-0-76.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 11 08:54:11 ip-10-0-0-76.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Sep 11 08:54:11 ip-10-0-0-76.ec2.internal httpd[28410]: Server configured, listening on: port 80
[root@ip-10-0-0-76 ~]#

```

It works!

```
[root@ip-10-0-0-76 ~]# chmod 400 linux-testing.pem
[root@ip-10-0-0-76 ~]# ssh -i linux-testing.pem ec2-user@10.0.1.68

#####
--      Amazon Linux 2023
--
--      https://aws.amazon.com/linux/amazon-linux-2023
--
#####

[ec2-user@ip-10-0-1-68 ~]$ sudo su -
[root@ip-10-0-1-68 ~]# sudo dnf update -y
Amazon Linux 2023 kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-1-68 ~]# yum install httpd -y
Last metadata expiration check: 0:00:40 ago on Fri Sep 11 09:00:49 2026.
No match for argument: httpd
Error: Unable to find a match: httpd
[root@ip-10-0-1-68 ~]# yum install httpd -y
Last metadata expiration check: 0:00:51 ago on Fri Sep 11 09:00:49 2026.
Dependencies resolved.

=====
Package                                Architecture      Version            Repository          Size
=====
Installing:
httpd                                   x86_64            2.4.68-1.amzn2023.0.1  amazonlinux         46 k
Installing dependencies:
apr                                     x86_64            1.7.5-1.amzn2023.0.4  amazonlinux         129 k
=====

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64          apr-util-1.6.5-1.amzn2023.0.1.x86_64      apr-util-1mdb-1.6.5-1.amzn2023.0.1.x86_64
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.68-1.amzn2023.0.1.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64      httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch  httpd-tools-2.4.68-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64        mailcap-2.1.49-3.amzn2023.0.3.noarch        mod_http2-2.0.42-1.amzn2023.0.1.x86_64
mod_lua-2.4.68-1.amzn2023.0.1.x86_64

Complete!
[root@ip-10-0-1-68 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service - /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-1-68 ~]# systemctl start httpd
[root@ip-10-0-1-68 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 09:02:25 UTC; 16s ago
     Docs: man:httpd.service(8)
    Main PID: 28292 (httpd)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
       Tasks: 177 (limit: 1059)
      Memory: 13.9M
         CPU: 73ms
    CGroup: /system.slice/httpd.service
            └─28292 /usr/sbin/httpd -DFOREGROUND
              └─28293 /usr/sbin/httpd -DFOREGROUND
                └─28294 /usr/sbin/httpd -DFOREGROUND
                  └─28300 /usr/sbin/httpd -DFOREGROUND
                    └─28316 /usr/sbin/httpd -DFOREGROUND

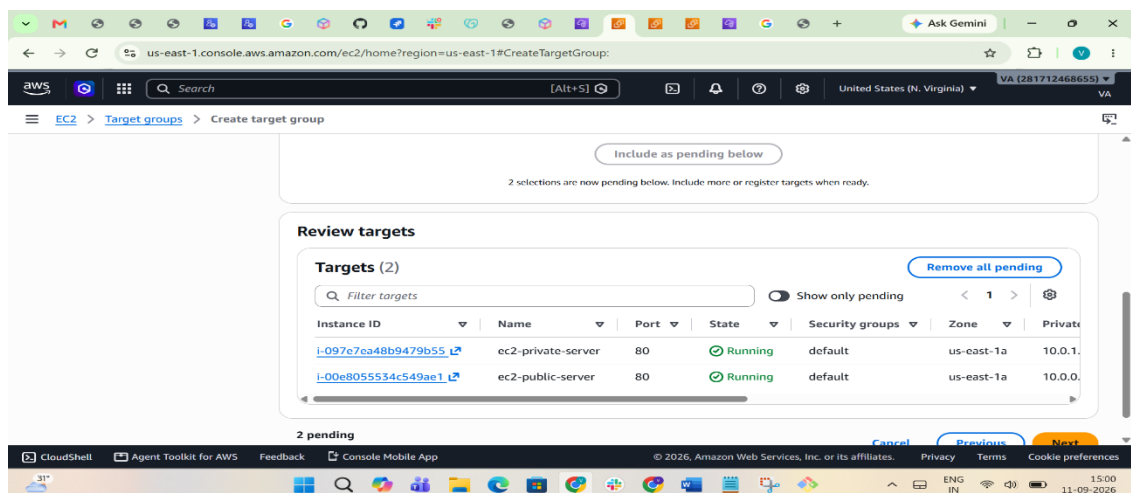
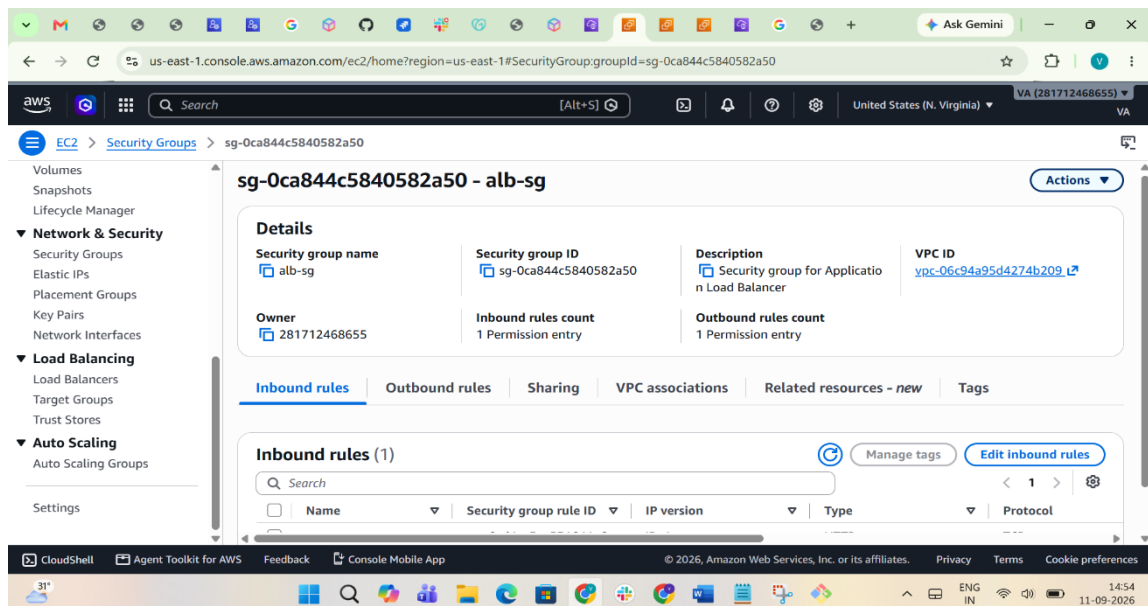
Sep 11 09:02:25 ip-10-0-1-68.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 11 09:02:25 ip-10-0-1-68.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Sep 11 09:02:25 ip-10-0-1-68.ec2.internal httpd[28292]: Server configured, listening on: port 80
[root@ip-10-0-1-68 ~]# curl 10.0.1.68
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
[root@ip-10-0-1-68 ~]# |
```

8. Create one application load balancer and attach it to both EC2 instances.

Implementation

1. Go to EC2 → Load Balancers.
2. Click **Create Load Balancer**.
3. Select **Application Load Balancer**.
4. Name: Apache-ALB.
5. Select **Internet-facing**.
6. Select My-vpc-virginia.
7. Select **two public subnets in different AZs**.
8. Configure ALB security group.
9. Allow HTTP 80.
10. Create a **Target Group**.
 - Target type: **Instances**.
 - Protocol: **HTTP**.

- Port: 80.
11. Configure health check path: /
 12. Register both Apache EC2 instances.
 13. Create the target group.
 14. Create an ALB listener:
 - Protocol: HTTP
 - Port: 80
 15. Forward to Apache target group.
 16. Wait for targets to become **Healthy**.
 17. Open the ALB DNS name in the browser.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#TargetGroup.targetGroupArn=arn:aws:elasticloadbalancing:us-east-1:281712468655:targetgroup/apache-target-group/b9081a1e38d1dadd

aws [Search] [Alt+S] United States (N. Virginia) VA (281712468655)

EC2 > Target groups > apache-target-group

Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Settings

apache-target-group

Actions

Details
arn:aws:elasticloadbalancing:us-east-1:281712468655:targetgroup/apache-target-group/b9081a1e38d1dadd

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-06c94a95d4274b209
IP address type IPv4	Load balancer None associated		

2 Total targets	0 Healthy	0 Unhealthy	2 Unused	0 Initial	0 Draining
--------------------	--------------	----------------	-------------	--------------	---------------

0 Anomalous

► **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

31° ENG IN 15:01 11-09-2026

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LoadBalancer.loadBalancerArn=arn:aws:elasticloadbalancing:us-east-1:281712468655:loadbalancer/application/my-application-alb

aws [Search] [Alt+S] United States (N. Virginia) VA (281712468655)

EC2 > Load balancers > my-application-alb

AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Introducing new Capacity Utilization metric for ALB
Monitor how much of your load balancer's available capacity was used to meet traffic demand. Visit the **Capacity** or **Monitoring** tabs to view utilization data.

my-application-alb

Actions

▼ **Details**

Load balancer type Application	Status Active	VPC vpc-06c94a95d4274b209	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z355XDOTRQ7X7K	Availability Zones subnet-0fdabd331e68229f2 us-east-1a (use1-az1) subnet-081815360a6591cf0 us-east-1b (use1-az2)	Date created September 11, 2026, 15:08 (UTC+05:30)

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

31° ENG IN 15:22 11-09-2026

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#TargetGroup.targetGroupArn=arn:aws:elasticloadbalancing:us-east-1:281712468655:targetgroup/apache-target-group/b9081a1e38d1dadd

aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) VA (281712468655)

EC2 > Target groups > apache-target-group

AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets Monitoring Health checks Attributes Tags

Registered targets (2) Info Anomaly mitigation: Not applicable Deregister Register targets

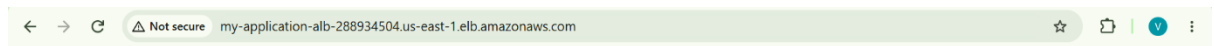
Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Q Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details
☐	i-097e7ea48b9479b55	ec2-private-se...	80	us-east-1a (us-east-1a)	Healthy	-
☐	i-00e8055534c549ae1	ec2-public-ser...	80	us-east-1a (us-east-1a)	Healthy	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

31° ENG IN 15:23 11-09-2026



9. Store application load balancer logs in S3.

Implementation

1. Go to **S3** → **Create bucket**.
2. Create a unique bucket.
3. Go to **EC2** → **Load Balancers**.
4. Select Apache-ALB.
5. Open **Attributes**.
6. Find **Access logs**.
7. Enable access logs.
8. Select the S3 bucket.
9. Save the configuration.
10. Generate traffic through the ALB.
11. Check the S3 bucket for Awslog files.

us-east-1.console.aws.amazon.com/s3/buckets/my-alb-access-logs-2026-0509?region=us-east-1&tab=objects

Successfully created bucket "my-alb-access-logs-2026-0509"

my-alb-access-logs-2026-0509

Objects (0)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
No objects				
You don't have any objects in this bucket.				

Upload

us-east-1.console.aws.amazon.com/s3/bucket/my-alb-access-logs-2026-0509/property/policy/edit?region=us-east-1

Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": "logdelivery.elasticloadbalancing.amazonaws.com"
8       },
9       "Action": "s3:PutObject",
10      "Resource": "arn:aws:s3:::my-alb-access-logs-2026-0509/AWSLogs/281712468655/*"
11    }
12  ]
13 }
```

Edit statement Remove

Add actions

Choose a service

Search services

Included

S3

Available

AI Operations

AMP

API Gateway

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LoadBalancerEditAttributes:loadBalancerArn=arn:aws:elasticloadbalancing:us-east-1:281712468655:loadbalancer/app/my-application-alb

my-application-alb

Disable - Default

Zonal shift will not be available to the load balancer.

Enable

Zonal shift will be available to the load balancer.

Protection

Deletion protection

To prevent your load balancer from being deleted accidentally, turn on deletion protection. If you turn on deletion protection, you must turn it off before you can delete the load balancer.

Deletion protection

Logging

All legacy logs are stored in S3. If no prefix is specified, logs are stored in the bucket root.

Access logs - legacy

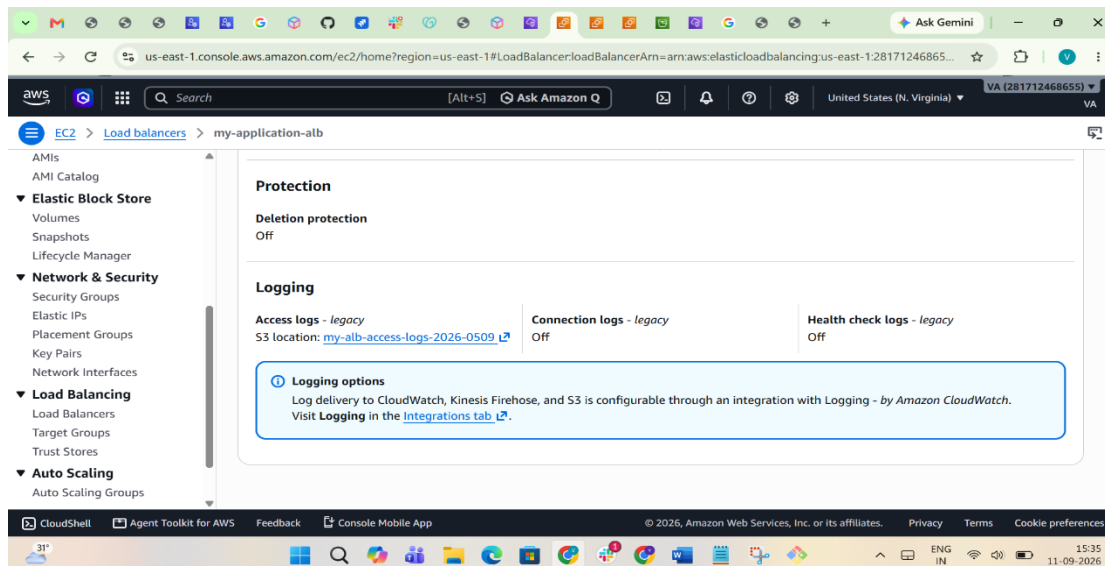
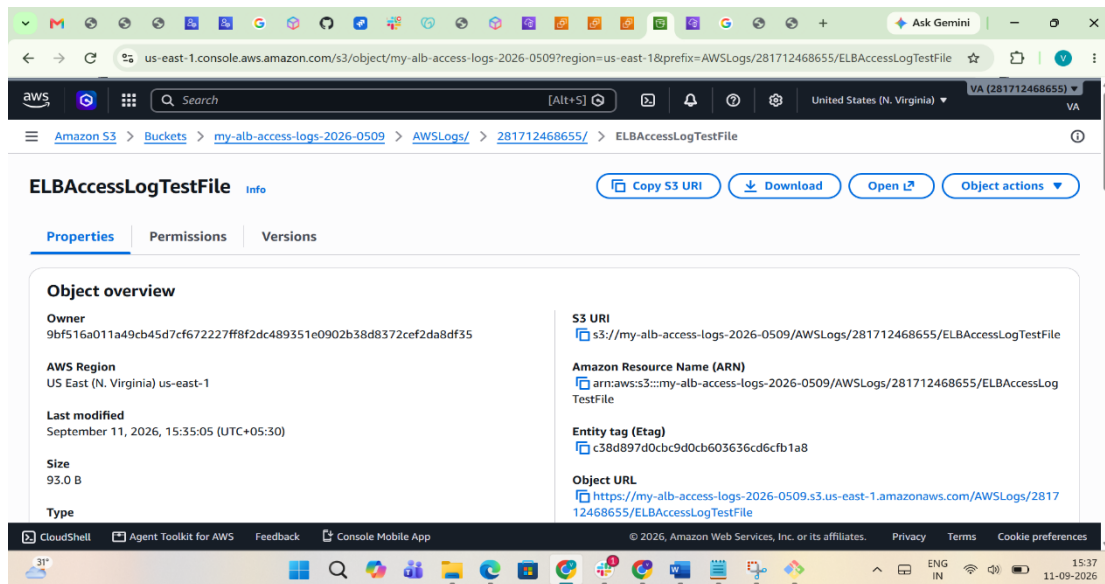
Access logs deliver detailed logs of all requests made to your load balancer. Additional charges apply. [Learn more](#)

S3 URI

s3://my-alb-access-logs-2026-0509

Format: s3://bucket/prefix

View S3



10. Store the VPC flow logs in a CloudWatch log group.

Implementation

1. Go to **VPC** → **Your VPCs**.
2. Select **My-Project-VPC**.
3. Choose **Actions** → **Create flow log**.
4. Select traffic type: **All**.
5. Select destination: **Send to CloudWatch Logs**.
6. Create a CloudWatch Log Group: **vpc-flowlogs**
7. Select/create the required IAM role.
8. Create the flow log.

9. Go to **CloudWatch** → **Logs** → **Log groups**.

10. Open `vpc-flowlogs`.

11. Verify that log streams and flow-log records are being generated.

The first screenshot shows the AWS Management Console page for a VPC flow log named 'my-flow-log-01' with ID 'fl-086a09157dd9ffca2'. The page is titled 'fl-086a09157dd9ffca2 / my-flow-log-01'. The left sidebar shows the 'VPC dashboard' with a 'Filter by VPC' dropdown and a list of VPC resources. The main content area displays the flow log details in a grid:

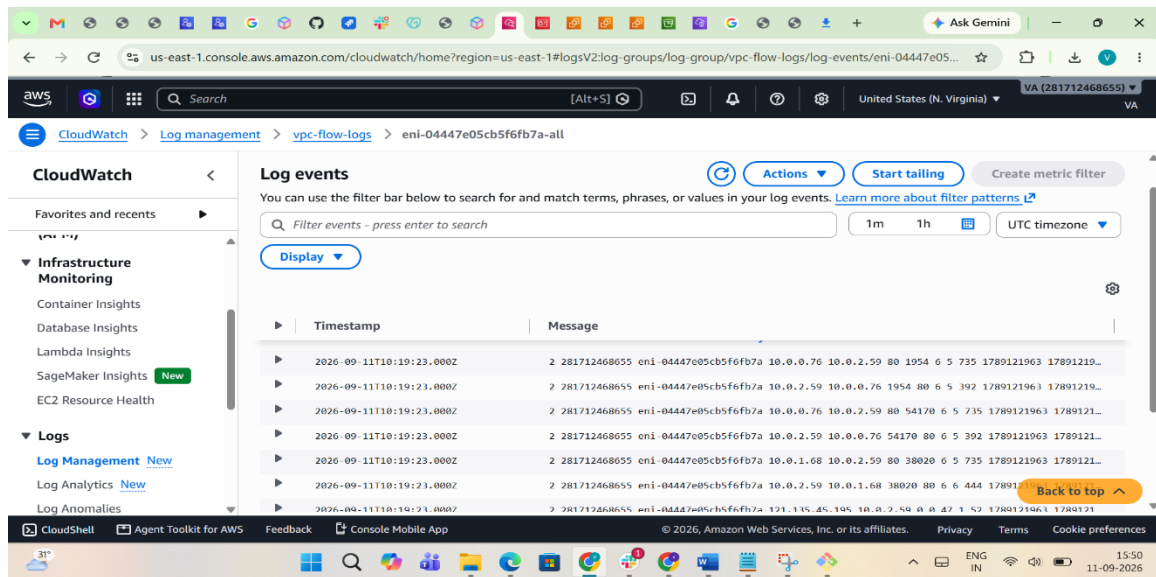
Details			
Flow Log ID fl-086a09157dd9ffca2	Destination Type cloud-watch-logs	Traffic Type All	File Format -
Name my-flow-log-01	Destination Name vpc-flow-logs	Max Aggregation Interval 1 minute	Hive Compatible Partitions -
State Active	IAM Role arn:aws:iam::281712468655:role/service-role/VPCFlowLogs-Cloudwatch-1789121885616	Log Format Default	Partition Logs -
Creation Time Friday, September 11, 2026 at 15:48:31 GMT+5:30	Cross Account IAM Role -	Tag field specifications -	

The second screenshot shows the AWS Management Console page for a VPC with ID 'vpc-06c94a95d4274b209'. The page is titled 'vpc-06c94a95d4274b209'. The left sidebar shows the 'VPC dashboard' with a 'Filter by VPC' dropdown and a list of VPC resources. The main content area displays the VPC details in a grid:

VPC Details			
Enabled default	Default VPC No	dopt-0c994995a0e6f7902	rtb-0b24fc265c8cb1e54 / public-RT
Main network ACL acl-0b62cab9656d8b0c9	Network Address Usage metrics Disabled	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Encryption control mode -	Route 53 Resolver DNS Firewall rule groups -	Owner ID 281712468655
Encryption control ID -			

Below the VPC details, there are tabs for 'Resource map', 'CIDRs', 'Flow logs', 'Tags', 'Related resources', and 'Integrations'. The 'Flow logs' tab is selected, showing a list of flow logs:

Name	Flow log ID	Traffic type
my-flow-log-01	fl-086a09157dd9ffca2	All



11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Implementation

15. Open **CloudWatch** → **Metrics**.

16. Select **EC2** → **Per-Instance Metrics**.

17. Select **CPUUtilization**.

18. Select the required EC2 instances.

19. Click **Add to dashboard**.

20. Create dashboard: **Ec2-Monitoring-Dashboard**

21. Add CPU utilization widgets.

Apache Service Monitoring

22. Create a Bash script to check Apache: `systemctl is-active --quiet httpd`

23. Publish:

- 1 → Apache running.
- 0 → Apache stopped.

24. Publish the value as a custom CloudWatch metric.

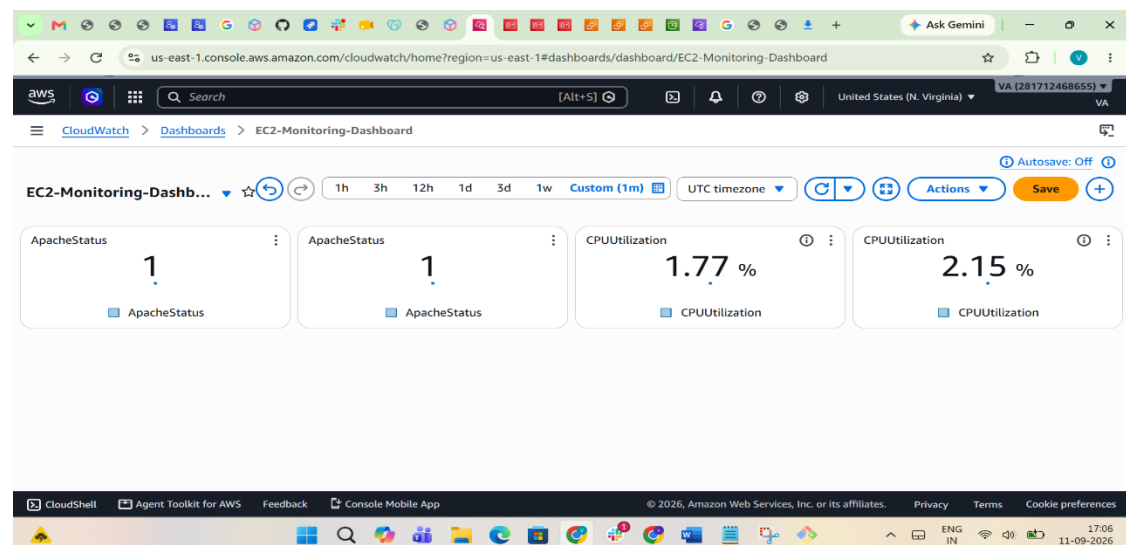
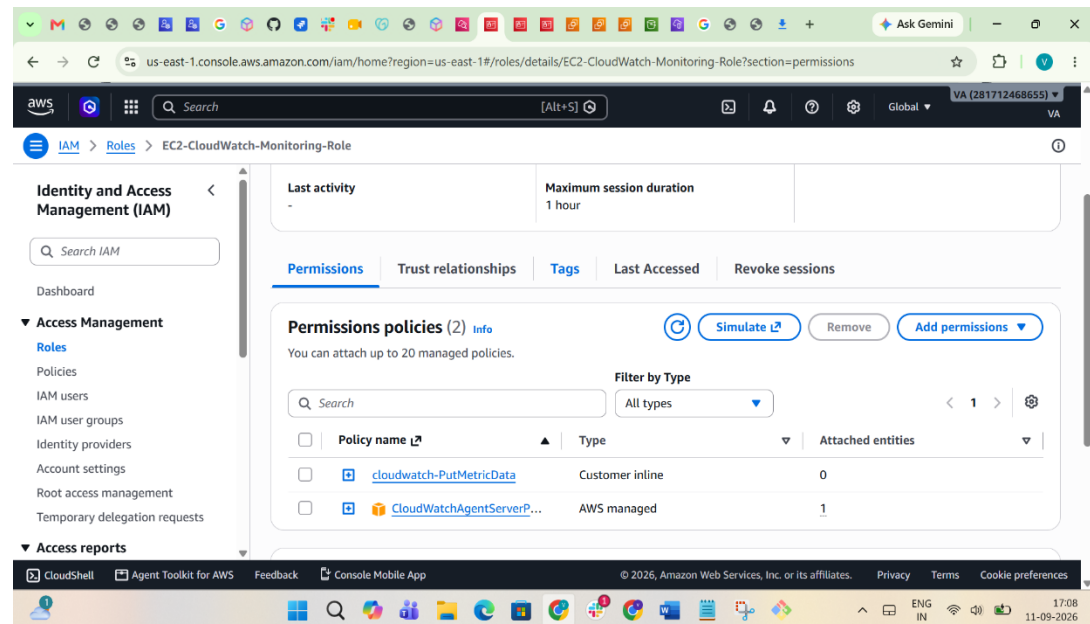
25. Namespace: **Custom/Apache**

26. Metric: **ApacheStatus**

27. Configure a cron job to run the script periodically.

28. Go to **CloudWatch** → **Metrics** → **Custom/Apache**.

29. Select ApacheStatus.
30. Add it to the dashboard.
31. Create an alarm for ApacheStatus = 0.

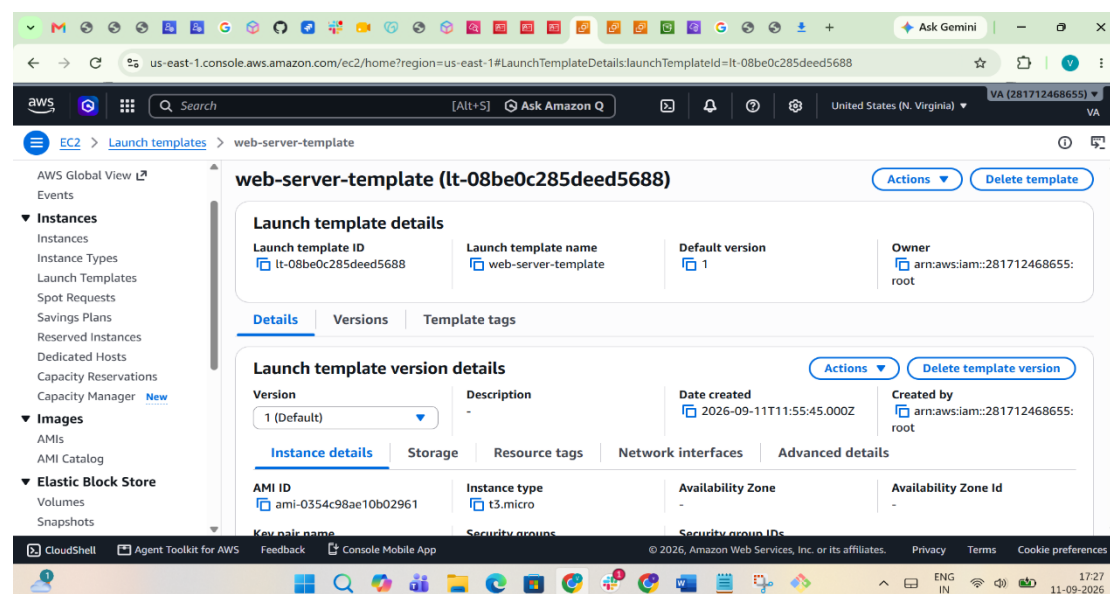


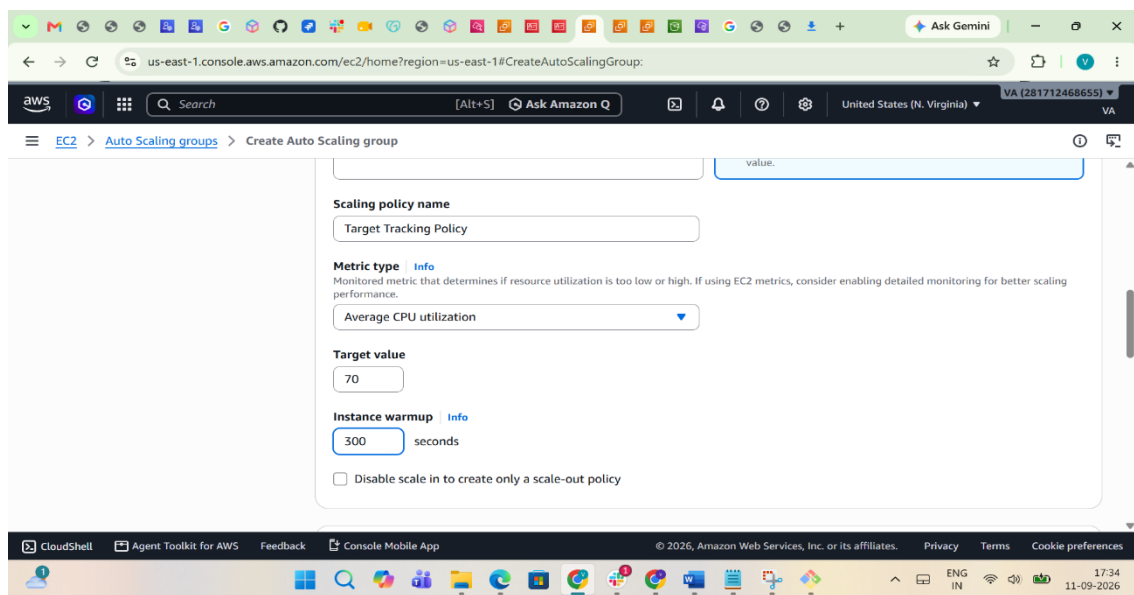
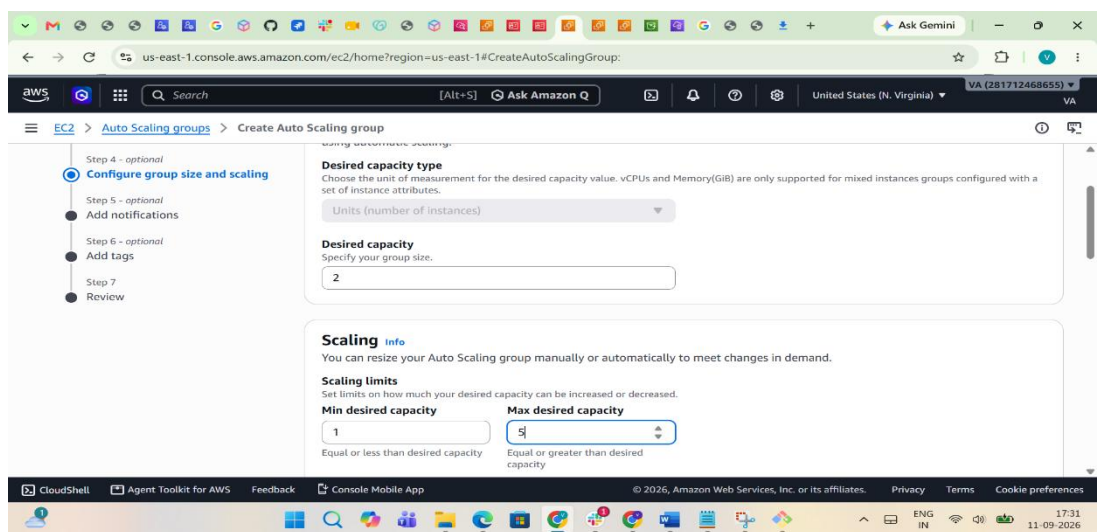
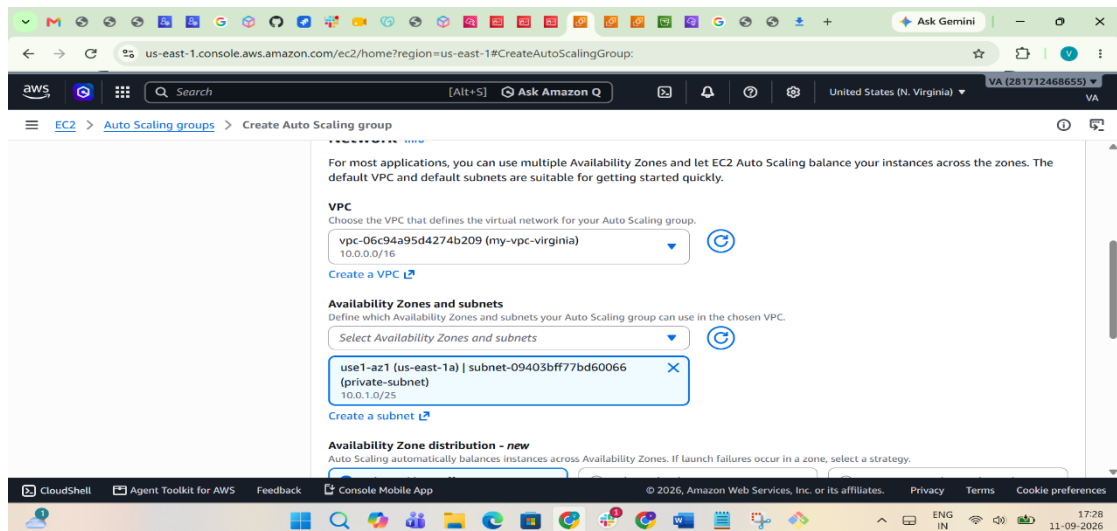
12.If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

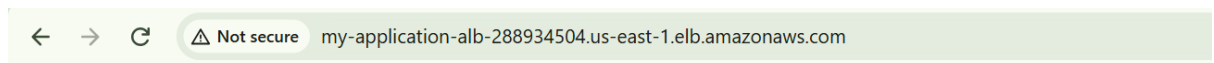
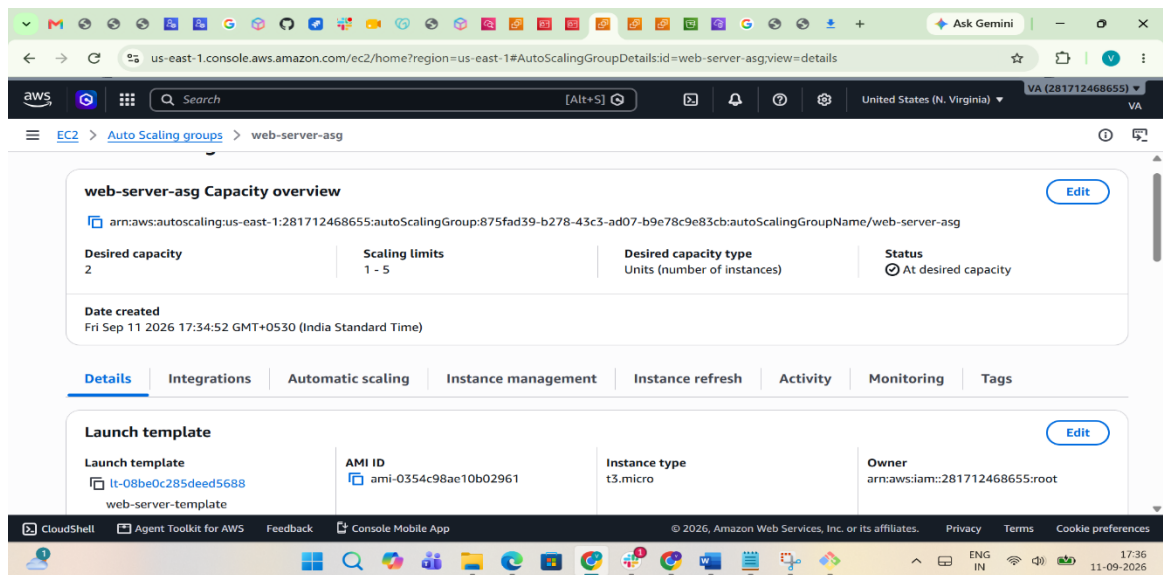
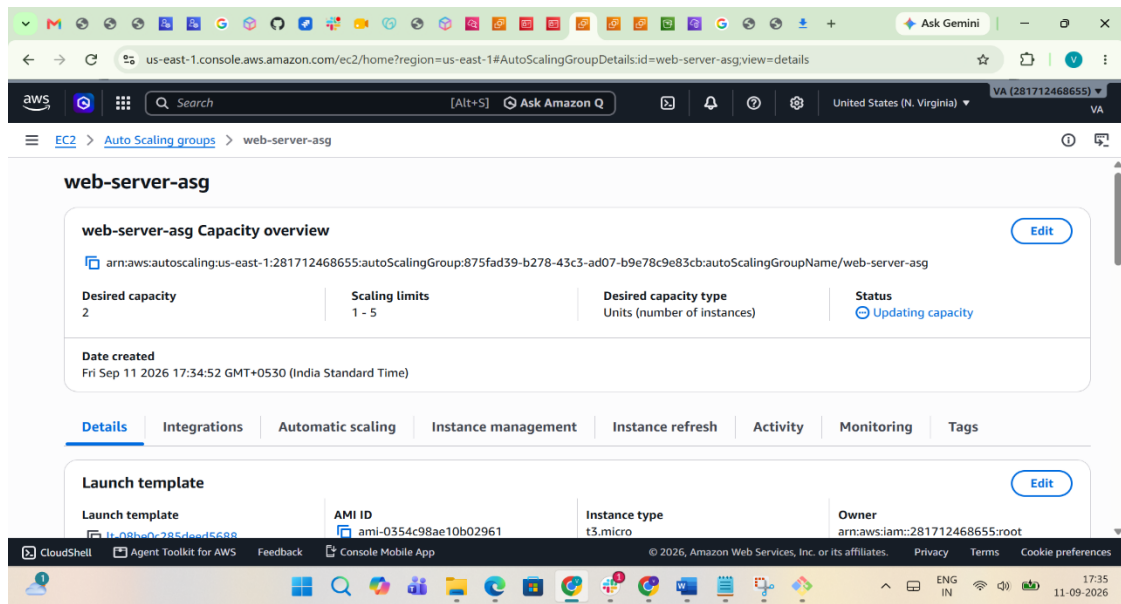
Implementation

1. Create an **EC2 Launch Template**.
2. Select Amazon Linux 2023.
3. Select the required instance type.
4. Configure the Apache security group.
5. Add User Data to automatically install and start Apache.
6. Create an **Auto Scaling Group**.

7. Select the Launch Template.
8. Select the VPC.
9. Select **private subnets in multiple AZs**.
10. Set:
11. Minimum capacity: 1
12. Desired capacity: 2
13. Maximum capacity: 5
14. Attach the Apache target group to the ASG.
15. Enable ELB health checks.
16. Create a **dynamic scaling policy**.
17. Select CPU utilization as the scaling metric.
18. Set the target/threshold to approximately **70%**, according to the assignment.
19. Configure the scaling policy to increase desired capacity by 2 when the CPU condition is met.
20. CloudWatch monitors CPU utilization.
21. When CPU exceeds the configured threshold, the alarm triggers the scaling policy.
22. Auto Scaling launches a new EC2 instance.
23. The new instance automatically installs Apache through User Data.
24. The new instance registers with the ALB target group.



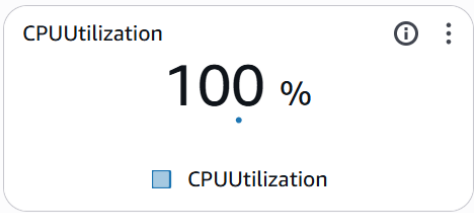
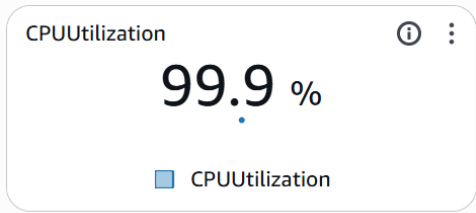




Hello from Auto Scaling EC2

Apache HTTP Server is running successfully.

This instance was launched by the Auto Scaling Group.



☐	i-0cc0365f0cbad806b	✓ Running 🔍 🔍	t3.micro	🕒 Initializing	us-east-1a
☐	i-06a42bab23426373e	✓ Running 🔍 🔍	t3.micro	🕒 Initializing	us-east-1a